

Security Audit

Spicenet (DeFi)

Table of Contents

Executive Summary	4
Project Context	4
Audit scope	7
Security Rating	8
Intended Smart Contract Functions	9
Code Quality	12
Audit Resources	12
Dependencies	12
Severity Definitions	13
Status Definitions	14
Audit Findings	15
Centralisation	30
Conclusion	31
Our Methodology	32
Disclaimers	34
About Hashlock	35

CAUTION

THIS DOCUMENT IS A SECURITY AUDIT REPORT AND MAY CONTAIN CONFIDENTIAL INFORMATION. THIS INCLUDES IDENTIFIED VULNERABILITIES AND MALICIOUS CODE WHICH COULD BE USED TO COMPROMISE THE PROJECT. THIS DOCUMENT SHOULD ONLY BE FOR INTERNAL USE UNTIL ISSUES ARE RESOLVED. ONCE VULNERABILITIES ARE REMEDIATED, THIS REPORT CAN BE MADE PUBLIC. THE CONTENT OF THIS REPORT IS OWNED BY HASHLOCK PTY LTD FOR USE OF THE CLIENT.

Executive Summary

The Spicenet team partnered with Hashlock to conduct a security audit of their smart contract. Hashlock manually and proactively reviewed the code in order to ensure the project's team and community that the deployed contracts are secure.

Project Context

Spicenet is a brokerage network in the DeFi (decentralized finance) space, aiming to simplify how applications, assets, and cross-chain ecosystems interact with liquidity, financial instruments, and yield sources. Their two main offerings — Spice Edge and Spice Flow — are SDK/API tools that let developers and projects tap into multiple DeFi venues (exchanges, liquidity pools, yield aggregators) with a single integration. The goal is to provide better execution, distribution, and UX (user experience) across chains, making it easier to scale apps, launch new assets, and deliver competitive financial outcomes.

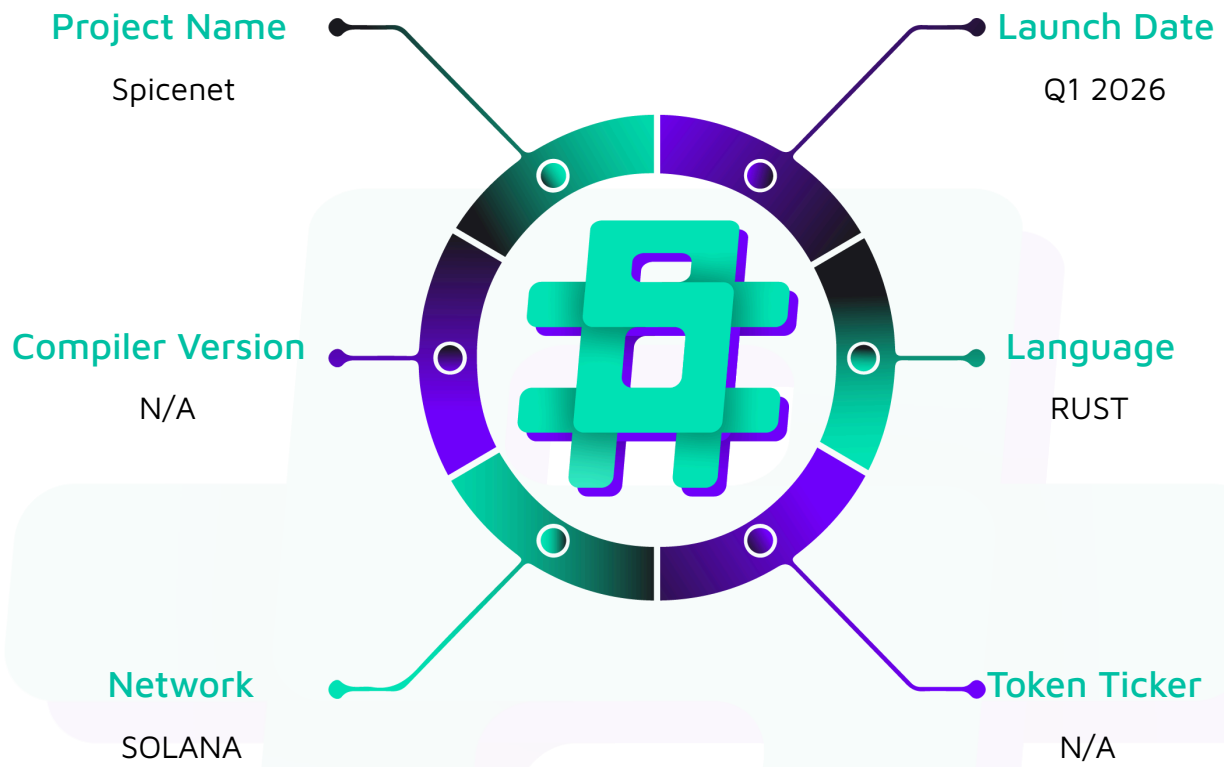
Project Name: Spicenet

Project Type: DeFi

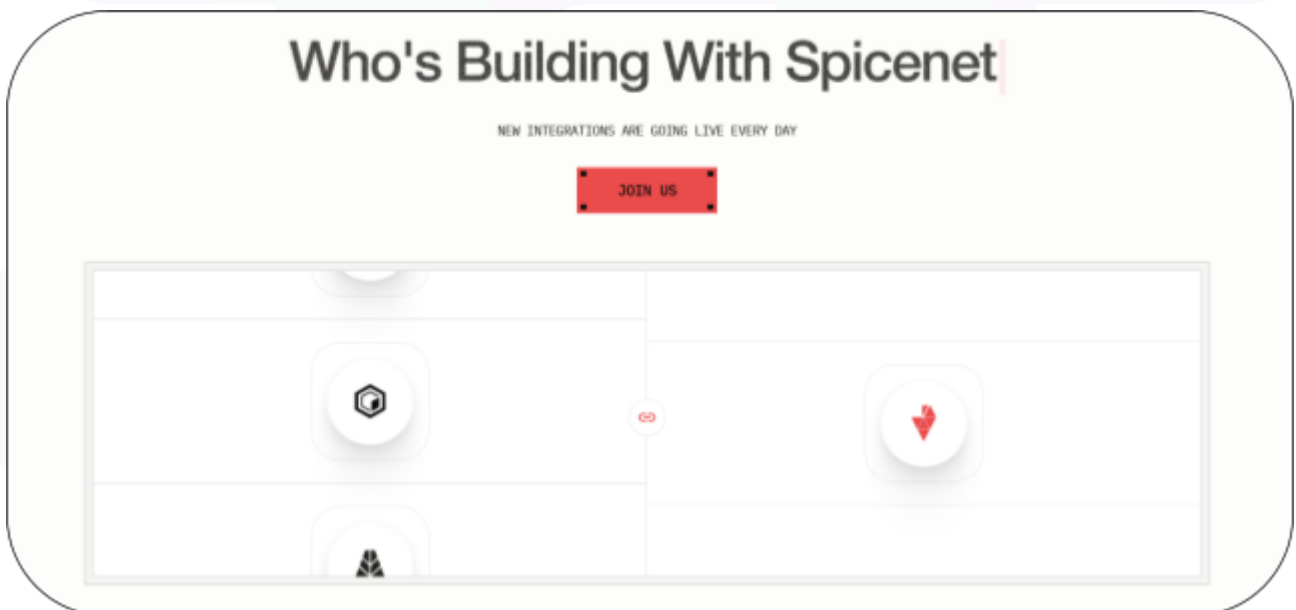
Website: <https://spicenet.io/>

Logo:



Visualised Context:

Project Visuals:



Audit scope

We at Hashlock audited the rust code within the Spicenet project. The scope of work included a comprehensive review of the smart contracts listed below. We tested the smart contracts to check for their security and efficiency. These tests were undertaken primarily through manual line-by-line analysis and were supported by software-assisted testing.

Description	Spicenet Smart Contracts
Platform	Spicenet / Rust
Audit Date	September, 2025
Crate 1	Time
Crate 2	Stf
Crate 4	Sokoban
Crate 5	Shared
Crate 6	Oracle
Crate 7	Rollup
Crate 8	Provers
Crate 9	Instruments
Crate 11	Capsule
GitHub Commit Hash	658f63d635e283c93e922fae0ae27c49e83d871a
Fix Review GitHub Commit Hash	7029d0ae5b6e87b5627d8491918cf6115c696e67

Security Rating

After Hashlock's Audit, we found the smart contracts to be **"Hashlocked"**. The contracts all follow correct logic, with proper and detailed ordering.



Not Secure

Vulnerable

Secure

Hashlocked

The 'Hashlocked' rating is reserved for projects that ensure ongoing security via bug bounty programs or on chain monitoring technology.

All issues uncovered during automated and manual analysis were meticulously reviewed and applicable vulnerabilities are presented in the [Audit Findings](#) section. The general security overview is presented in the [Standardised Checks](#) section and the project's contract functionality is presented in the [Intended Smart Contract Functions](#) section.

All vulnerabilities initially identified have now been resolved and acknowledged.

Hashlock found:

11 High severity vulnerabilities

3 Medium severity vulnerabilities

Caution: Hashlock's audits do not guarantee a project's success or ethics, and are not liable or responsible for security. Always conduct independent research about any project before interacting.

Intended Smart Contract Functions

Claimed Behaviour	Actual Behaviour
Capsule crate: - Manage Admin, Recovery and Ephemeral wallets - Verify signatures and scope permissions - Dispatch Execute calls to DEX - Multisig propose, vote, timelock and execute	Contract achieves this functionality.
Instruments crate: - Define spot and derivative products - Configure ticks, variance, and params - Close or update product metadata - Link products to MPGs	Contract achieves this functionality.
Oracle crate: - Register and whitelist Oracle nodes - Manage stakes and exits - Track node roles and public keys	Contract achieves this functionality.
Sokoban crate: - Ingest and queue transactions - Prioritize and schedule execution - Build blocks within resource limits - Run periodic maintenance jobs	Contract achieves this functionality.

stf crate: - Apply transactions deterministically - Enforce nonce, fee, and gas/weight rules - Dispatch to module entrypoints - Emit state roots and events	Contract achieves this functionality.
Rollup crate: - Batch L2 blocks for L1 and DA - Compute commitments and inclusions - Handle finality windows and challenges - Coordinate proof submission	Contract achieves this functionality.
Provers crate: - Generate proofs from STF traces - Aggregate per-tx or per-block proofs - Output verifier inputs for Rollup - Manage proving and verification keys	Contract achieves this functionality.
time crate: - Provide chain time, slots, and epoch math - Validate transaction timestamp windows and expiries - Compute timelocks and unlock times for governance flows - Gate periodic tasks and maintenance windows	Contract achieves this functionality.

shared crate: - Define common domain types, IDs, and enums - Provide serialization, hashing, and message-domain helpers - Centralize errors, result codes, and config constants - Offer numeric utilities for fixed-point and safe arithmetic - Share reusable collections and small algorithms across crates	Contract achieves this functionality.
--	--

Code Quality

This audit scope involves the smart contracts of the Spicenet project, as outlined in the Audit Scope section. All contracts, libraries, and interfaces mostly follow standard best practices and to help avoid unnecessary complexity that increases the likelihood of exploitation, however, some refactoring was required.

The code is very well commented on and closely follows best practice nat-spec styling. All comments are correctly aligned with code functionality.

Audit Resources

We were given the Spicenet project smart contract code in the form of Github access.

As mentioned above, code parts are well commented. The logic is straightforward, and therefore it is easy to quickly comprehend the programming flow as well as the complex code logic. The comments are helpful in providing an understanding of the protocol's overall architecture.

Dependencies

As per our observation, the libraries used in this smart contracts infrastructure are based on well-known industry standard open source projects.

Apart from libraries, its functions are used in external smart contract calls.

Severity Definitions

The severity levels assigned to findings represent a comprehensive evaluation of both their potential impact and the likelihood of occurrence within the system. These categorizations are established based on Hashlock's professional standards and expertise, incorporating both industry best practices and our discretion as security auditors. This ensures a tailored assessment that reflects the specific context and risk profile of each finding.

Significance	Description
High	High-severity vulnerabilities can result in loss of funds, asset loss, access denial, and other critical issues that will result in the direct loss of funds and control by the owners and community.
Medium	Medium-level difficulties should be solved before deployment, but won't result in loss of funds.
Low	Low-level vulnerabilities are areas that lack best practices that may cause small complications in the future.
Gas	Gas Optimisations, issues, and inefficiencies
QA	Quality Assurance (QA) findings are informational and don't impact functionality. Supports clients improve the clarity, maintainability, or overall structure of the code.

Status Definitions

Each identified security finding is assigned a status that reflects its current stage of remediation or acknowledgment. The status provides clarity on the handling of the issue and ensures transparency in the auditing process. The statuses are as follows:

Significance	Description
Resolved	The identified vulnerability has been fully mitigated either through the implementation of the recommended solution proposed by Hashlock or through an alternative client-provided solution that demonstrably addresses the issue
Acknowledged	The client has formally recognized the vulnerability but has chosen not to address it due to the high cost or complexity of remediation. This status is acceptable for medium and low-severity findings after internal review and agreement. However, all high-severity findings must be resolved without exception.
Unresolved	The finding remains neither remediated nor formally acknowledged by the client, leaving the vulnerability unaddressed.

Audit Findings

High

[H-01] `crates/capsule/src/lib.rs#call` - Unenforced nonce allows replay attacks

Description

The `nonce` parameter is included in the signed message, which is intended to prevent replay attacks when verifying the signatures.

However, there is no validation implemented to ensure uniqueness. Specifically, the entry points do not verify that the nonce is greater than the last used value (in an incrementing ordering) or that it has not been previously consumed.

As a result, any previously valid signed message can be replayed indefinitely. For example, an attacker can reuse the signatures from other users and resubmit them, causing repeated execution of the same operation.

Impact

Signed messages can be replayed multiple times. For example, the attacker can repeatedly replay the `CapsuleMessageType::Withdraw` call to withdraw funds from the `dex` module.

Recommendation

- Track nonce per address in state.
- Reject messages where `nonce <= last_used_nonce`.

Status

Resolved

[H-02] crates/capsule/src/state/wallet.rs#verify_wallet_is_valid_transacting

- Missing binding validation between approving wallet and capsule wallet

Description

The `verify_wallet_is_valid_transacting` function checks whether a wallet is an admin or a valid ephemeral wallet for the current timestamp and scope.

However, it does not verify that the wallet (`approving_wallet`) belongs to the capsule smart wallet (`capsule_address`) being executed, allowing authentication bypass.

For example, an attacker can create their own ephemeral wallet with the correct scope (e.g., `Scope::Trg`) and use it to execute transactions on other users' capsules, bypassing ownership checks.

Impact

Attackers can perform privileged actions against another user's capsule. This results in authorization bypass and direct control of another user's smart wallet.

Recommendation

Implement the `self.verify_wallet_exists(wallet)` validation in the `verify_wallet_is_valid_ephemeral` function in `crates/capsule/src/state/wallet.rs:285` as `role_match` `&& wallet.is_active(timestamp) && self.verify_wallet_exists(wallet)`.

Status

Resolved

[H-03] `crates/capsule/src/call/multisig.rs#multisig_propose` - Proposal overwriting via attacker-controlled seed

Description

The `multisig_propose` function generates the proposal ID using the `get_proposal_id` helper function, where the seed is fully caller-controlled input and hashed as `(b"proposal", multisig_address, seed)` to derive the proposal ID.

This means a malicious multisig member can deliberately specify a seed that collides with (or exactly reuses) an existing proposal ID, causing the original proposal to be overwritten silently.

Impact

Any multisig member with the `Propose` scope can replace an existing proposal of another member. This allows proposal hijacking, where the contents can be replaced with malicious payloads while reusing the same ID.

Recommendation

Validate that the proposal ID does not exist.

Status

Resolved

[H-04] `crates/capsule/src/call/multisig.rs#multisig_execute` - Proposals can be repeatedly executed

Description

The `multisig_execute` function enforces that a proposal's status is `Approved` before executing the proposal.

However, the proposal's status is never updated to `CapsuleMultisigProposalStatus::Executed` state after execution. This means that the proposal will remain in the `Approved` state, allowing repeated calls to the same proposal.

Impact

Any approved proposals can be replayed indefinitely, executing the same action multiple times.

Recommendation

Update the proposal's status to `CapsuleMultisigProposalStatus::Executed` state.

Status

Resolved

[H-05] crates/capsule/src/call/wallet_management.rs#revoke_wallet -

Broken authorization logic blocks wallet revocations

Description

The `revoke_wallet` function implements the validation as `!smart_wallet.verify_wallet_is_admin(&approving_wallet) || !smart_wallet.verify_wallet_is_valid_recovery(&approving_wallet)` to authenticate the approving wallet. This is incorrect because the approving wallet must satisfy both conditions (being an admin and a recovery wallet simultaneously), which is not possible.

Impact

The `revoke_wallet` function cannot be called, preventing users from revoking wallets.

Recommendation

Modify the condition to be `!smart_wallet.verify_wallet_is_valid_recovery(&wallet_type)` so it ensures the `wallet_type` is a recovery wallet. Additionally, consider updating the error message to reflect this condition.

Status

Resolved

[H-06] `crates/capsule/src/state/multisig.rs#check_scopes` - Incorrect scope validation logic

Description

The `check_scopes` function performs the scope validation as `self.get_scopes(address).iter().all(|scope| scopes.contains(scope))`. This is problematic because:

- If a member has no scopes, `.all()` on an empty iterator evaluates to `true`, incorrectly granting authorization.
- If a member has more scopes than required, the validation fails incorrectly. Example: a member with `[Execute, Propose]` will fail when only `Execute` is required, as `.all()` enforces that all of the member's scopes must be in the requested set.

Impact

1. Unauthorized members with no scopes may gain access.
2. Legitimate members with multiple scopes may be denied.

In both cases, the intended access control guarantees of the multisig system is broken.

Recommendation

Update logic to validate required scopes properly:

```
scopes.iter().all(|required| self.get_scopes(address).contains(required))
```

Status

Resolved

[H-07] `crates/capsule/src/state/wallet.rs#create_wallet` - Non-deterministic wallet address generation

Description

The `create_wallet` function generates the wallet address using `OsRng`, a non-deterministic source of randomness. This is because different validators may derive different wallet addresses from the same transaction input, resulting in non-determinism and halting the chain.

Impact

The chain will halt, causing a denial of service issue.

Recommendation

Remove usage of `OsRng` for wallet address generation and derive wallet addresses deterministically.

Status

Resolved

[H-8] crates/oracle/registry/src/call.rs#register - Replay of `register` allows overwriting oracle node records

Description

The `register` function allows a whitelisted user to become an oracle node by bonding tokens. However, the function blindly overwrites the node state without validating that the `node_address` key does not exist in `self.oracle_nodes`.

Impact

A whitelisted user can overwrite another node's penalty and staked amount, and replace the existing whitelisted user address. This causes a loss of funds for the other node, as they cannot withdraw their leftover funds.

Recommendation

Implement a validation to return an error if the node already exists.

Status

Resolved

[H-9] `crates/oracle/registry/src/call.rs#exit` - Drainage of node stakes amount due to missing binding between `node_address` and `user_address`

Description

The `exit` function lets a user withdraw their bonded stake when leaving as an oracle node. However, there is no validation that the provided `node_address` actually belongs to the calling `user_address`.

Since `oracle_node` is fetched based on the provided `node_address`, and its funds are transferred to the caller, a malicious user can perform the following actions:

1. Provide their own `user_address`.
2. Pass in another oracle's `node_address`, which is the node from which they want to steal the funds.
3. Receive that node's bonded funds, while simultaneously deleting the victim's node entry via `self.oracle_nodes.remove(&node_address, state)`.

Impact

Any whitelisted user can drain any other oracle node's stake. This attack can be repeatedly executed via `exit() -> register() -> exit() -> register()` routes to steal funds from all nodes.

Recommendation

Bind the user address into the `OracleNode` struct during registration. Upon `exit`, ensure that the `user_address` is the registered address for the `oracle_node`.

Status

Resolved

[H-10]

`crates/capsule/src/call/wallet_management.rs#add_admin_wallet,add_recovery_wallet,add_ephemeral_wallet` - No proof-of-control and global uniqueness enable address squatting

Description

Adding a new admin, recovery, or ephemeral wallet requires only the approving wallet's signature. The newly added wallet is never required to co-sign to prove it controls the key.

Separately, `throw_duplicate_wallet_error` enforces system-wide uniqueness so a single external wallet cannot belong to multiple capsules. Together, this lets an attacker add a victim's EOA to their own capsule and "reserve" it system-wide.

Impact

Victims are blocked from using their own address to create or join a capsule. If revoke is unavailable, this becomes a long-lived denial of service.

Recommendation

Require a co-signature from the wallet being added to prove control. Consider relaxing uniqueness from global to per-capsule. If global uniqueness must stay, add an explicit "claim" flow that proves control on first association.

Status

Resolved

[H-11] crates/capsule/src/call/wallet_management.rs#revoke_wallet -

Missing capsule scoping allows cross-capsule revocations

Description

The `revoke_wallet` fetches the wallet by its `WalletType` and flips the `revoked` flag. It does not assert that the wallet belongs to the capsule provided in the call.

Once the broken OR-check is fixed, an admin from Capsule A could revoke a wallet that actually belongs to Capsule B if they know the type.

```
pub fn revoke_wallet(
    &mut self,
    capsule_address: S::Address,
    approving_wallet_type: WalletType,
    wallet_type: WalletType,
    signature: Vec<u8>,
    nonce: u64,
    _context: &Context<S>,
    state: &mut impl TxState<S>,
) -> Result<()> {
    let approving_wallet = self
        .wallets
        .get(&approving_wallet_type, state)?
        .ok_or(anyhow::anyhow!("Wallet not found"))?;

    let smart_wallet = self
        .smart_wallets
        .get(&capsule_address, state)?
        .ok_or(anyhow::anyhow!("Smart wallet not found"))?;

    if !smart_wallet.verify_wallet_is_admin(&approving_wallet)
        || !smart_wallet.verify_wallet_is_valid_recovery(&approving_wallet)
    {
        bail!("Approving wallet is not an admin");
    }
}
```

```

    }

    let message =
        WalletState::<S>::revoke_wallet_message(&capsule_address, &wallet_type,
nonce)?;

    if !approving_wallet.verify_signature(signature.as_ref(), &message)? {
        bail!("Invalid signature");
    }

    let mut wallet = self.wallets.get(&wallet_type, state)?.unwrap();
    wallet.revoked = true;
    self.wallets.set(&wallet_type, &wallet, state)?;

```

Impact

Cross-tenant denial of service by revoking wallets in other capsules.

Recommendation

After loading the wallet, assert `wallet.smart_wallet == capsule_address` before mutating. Abort if it does not match.

Status

Resolved

Medium

[M-01] [crates/capsule/src/call/multisig.rs#initialize_multisig](#) - Weak validation of multisig members and threshold

Description

The `initialize_multisig` function lacks two crucial validation checks:

1. Duplicate members are not deduped

The `members` vector is accepted as-is, without enforcing uniqueness. If the same address is inserted multiple times, the member count will be inflated.

2. Zero thresholds are not rejected

The function checks that `threshold > members.len()` is invalid, but does not forbid zero threshold values. A multisig initialized with zero thresholds would incorrectly allow execution of any proposals without approvals, defeating the purpose of a multisig.

Impact

- Duplicate members compromise the security of the multisig, causing it not to work as intended.
- A zero threshold results in a trivially bypassed multisig, effectively removing approval requirements.

Recommendation

- Ensure members contain unique addresses before creating the multisig.
- Enforce that `threshold >= 1 && threshold <= members.len()`.

Status

Resolved

[M-02] crates/capsule/src/state/wallet.rs#verify_wallet_is_valid_ephemeral**- Overly strict scope equality validation****Description**

The `verify_wallet_is_valid_ephemeral` function validates the scope via an exact equality check (`wallet_scopes == scopes`). This is problematic because if the wallet has a superset of scopes (e.g., `[Trading, Funds]`) but the call only requires one of the scopes (e.g., `[Trading]`), the check incorrectly fails.

Impact

Valid ephemeral wallets are rejected incorrectly.

Recommendation

Replace equality check with a containment-based check using the existing `contains()` helper function from `crates/capsule/src/state/wallet.rs:210`.

Status

Resolved

[M-03] `crates/oracle/registry/src/call.rs#whitelist` - Replay of `whitelist` overwrites existing user record

Description

The `whitelist` function allows the designated `registry_authority` to add an address to the whitelist. However, it blindly overwrites existing records and does not validate that the address is already whitelisted. If the user was previously registered (i.e., `is_oracle_node: true`), a repeated call will reset it back to `false`.

Impact

The whitelisted user that is tied to the specific node will incorrectly show `is_oracle_node: false`.

Recommendation

Implement a validation to return an error if the user is already whitelisted.

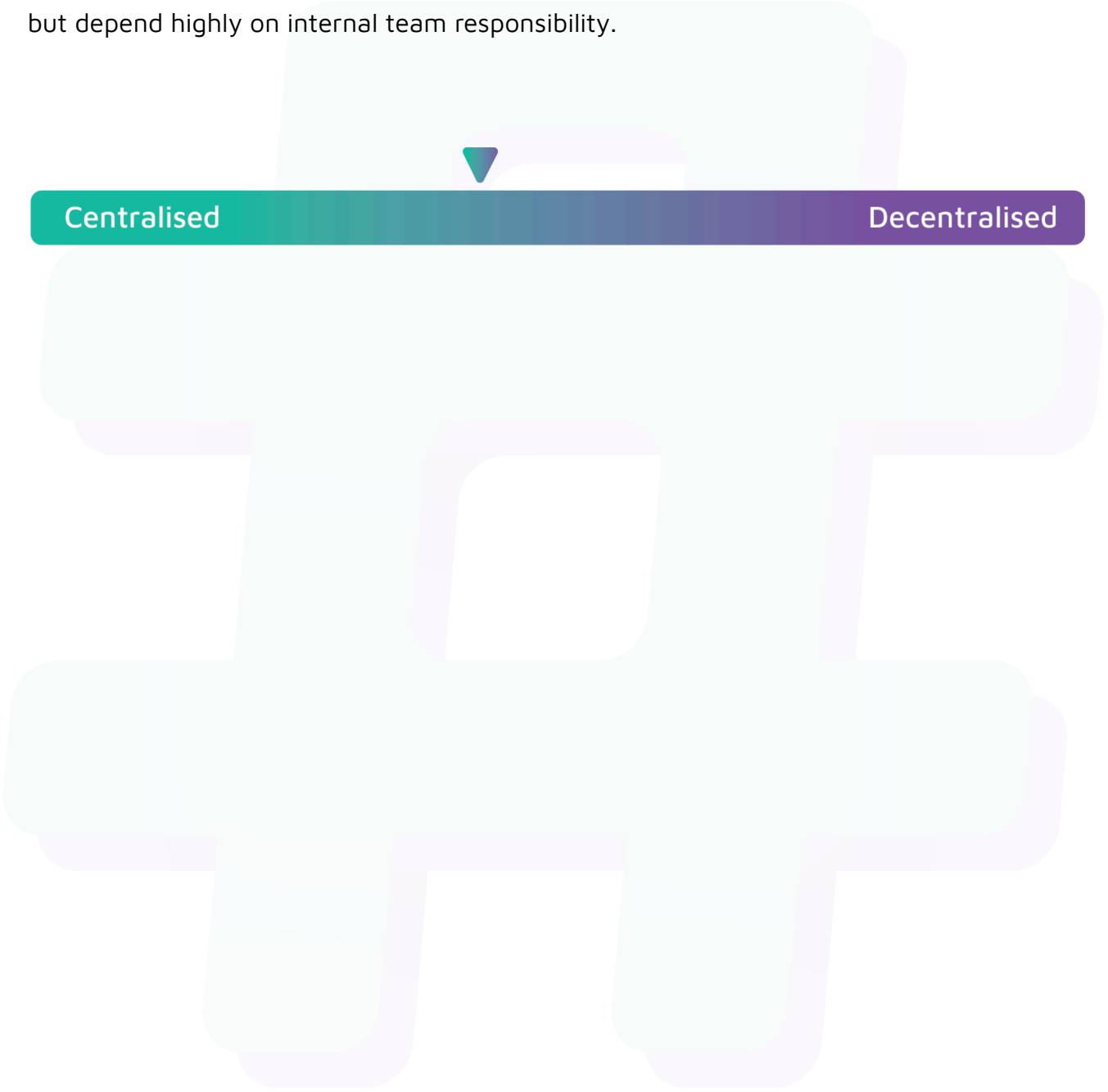
Status

Resolved

Centralisation

The Spicenet project values security and utility over decentralisation.

The owner executable functions within the protocol increase security and functionality but depend highly on internal team responsibility.



Centralised

Decentralised

Conclusion

After Hashlock's analysis, the Spicenet project seems to have a sound and well-tested code base, now that our vulnerability findings have been resolved and acknowledged. Overall, most of the code is correctly ordered and follows industry best practices. The code is well commented on as well. To the best of our ability, Hashlock is not able to identify any further vulnerabilities.

Our Methodology

Hashlock strives to maintain a transparent working process and to make our audits a collaborative effort. The objective of our security audits is to improve the quality of systems and upcoming projects we review and to aim for sufficient remediation to help protect users and project leaders. Below is the methodology we use in our security audit process.

Manual Code Review:

In manually analysing all of the code, we seek to find any potential issues with code logic, error handling, protocol and header parsing, cryptographic errors, and random number generators. We also watch for areas where more defensive programming could reduce the risk of future mistakes and speed up future audits. Although our primary focus is on the in-scope code, we examine dependency code and behaviour when it is relevant to a particular line of investigation.

Vulnerability Analysis:

Our methodologies include manual code analysis, user interface interaction, and white box penetration testing. We consider the project's website, specifications, and whitepaper (if available) to attain a high-level understanding of what functionality the smart contract under review contains. We then communicate with the developers and founders to gain insight into their vision for the project. We install and deploy the relevant software, exploring the user interactions and roles. While we do this, we brainstorm threat models and attack surfaces. We read design documentation, review other audit results, search for similar projects, examine source code dependencies, skim open issue tickets, and generally investigate details other than the implementation.

Documenting Results:

We undergo a robust, transparent process for analysing potential security vulnerabilities and seeing them through to successful remediation. When a potential issue is discovered, we immediately create an issue entry for it in this document, even though we have not yet verified the feasibility and impact of the issue. This process is vast because we document our suspicions early even if they are later shown to not represent exploitable vulnerabilities. We generally follow a process of first documenting the suspicion with unresolved questions, and then confirming the issue through code analysis, live experimentation, or automated tests. Code analysis is the most tentative, and we strive to provide test code, log captures, or screenshots demonstrating our confirmation. After this, we analyse the feasibility of an attack in a live system.

Suggested Solutions:

We search for immediate mitigations that live deployments can take and finally, we suggest the requirements for remediation engineering for future releases. The mitigation and remediation recommendations should be scrutinised by the developers and deployment engineers, and successful mitigation and remediation is an ongoing collaborative process after we deliver our report, and before the contract details are made public.

Disclaimers

Hashlock's Disclaimer

Hashlock's team has analysed these smart contracts in accordance with the best industry practices at the date of this report, in relation to: cybersecurity vulnerabilities and issues in the smart contract source code, the details of which are disclosed in this report, (Source Code); the Source Code compilation, deployment, and functionality (performing the intended functions).

Due to the fact that the total number of test cases is unlimited, the audit makes no statements or warranties on the security of the code. It also cannot be considered as a sufficient assessment regarding the utility and safety of the code, bug-free status, or any other statements of the contract. While we have done our best in conducting the analysis and producing this report, it is important to note that you should not rely on this report only. We also suggest conducting a bug bounty program to confirm the high level of security of this smart contract.

Hashlock is not responsible for the safety of any funds and is not in any way liable for the security of the project.

Technical Disclaimer

Smart contracts are deployed and executed on a blockchain platform. The platform, its programming language, and other software related to the smart contract can have their own vulnerabilities that can lead to attacks. Thus, the audit can't guarantee the explicit security of the audited smart contracts.

About Hashlock

Hashlock is an Australian-based company aiming to help facilitate the successful widespread adoption of distributed ledger technology. Our key services all have a focus on security, as well as projects that focus on streamlined adoption in the business sector.

Hashlock is excited to continue to grow its partnerships with developers and other web3-oriented companies to collaborate on secure innovation, helping businesses and decentralised entities alike.

Website: hashlock.com.au

Contact: info@hashlock.com.au



#hashlock.